

Update Packages in Python Poetry

By [Rex Resurreccion](#) Mar 03, 2022



This tutorial will discuss the steps on how to safely update packages in Python Poetry.

If you have not used Poetry before or you need to install this in your project, please read first my tutorial about [Python Poetry Cheat Sheet](#).

Verify Outdated Packages in Poetry

First step is to find out what packages are currently installed in your system. Then, find the ones that are outdated. You can easily show this by running a command `poetry show`.

1. poetry show -l

pytest	7.0.1	7.0.1
pytest-asyncio	0.18.1	0.18.1
pytest-cov	3.0.0	3.0.0
pytest-mock	3.7.0	3.7.0
python-dotenv	0.19.2	0.19.2
python-jose	3.3.0	3.3.0
python-json-logger	2.0.2	2.0.2
pytz	2021.3	2021.3
pyyaml	6.0	6.0
requests	2.26.0	2.27.1
rsa	4.8	4.8
sentry-asgi	0.2.0	0.2.0
sentry-sdk	1.5.6	1.5.6
six	1.16.0	1.16.0
smap	5.0.0	5.0.0
sniffio	1.2.0	1.2.0
starlette	0.17.1	0.18.0
starlette-context	0.3.3	0.3.3
starlette-prometheus	0.8.0	0.9.0

The `-l` flag will include the latest version available for each packages installed in your system. You can see at the second column the current version of your package. And at the third column is the latest version. You will also notice that the color is green if your package is up to date, yellow and red if there is an available update.

It sure looks like a daunting task updating your system at a first glance. With all the shown packages, how do we know where to start? Really, most of the listed packages from the command above are just chain of dependencies. Now, remember that Poetry keeps a master list of all the packages that you have explicitly installed. That file is `pyproject.toml`. My advise is to look at this file for starter.

Hello World ...

Search

Developer Tools

- [Code Editor](#)
- [Posted Codes](#)
- [Password Generator](#)
- [SSH Key Generator](#)
- [Subnet Calculator](#)
- [What Is My IP?](#)
- [Format IP Address](#)
- [WYSIWYG Editor](#)

Topics

- [Computer Hardware](#)
- [Featured](#)
- [Programming](#)
- [Tutorials](#)

This website uses cookies to improve your experience. By continuing to use YippeeCode.com you agree to its [Terms](#)

[of Service](#). ACCEPT

Update Packages in Poetry

Poetry provides a command `poetry update` for this task. Although, the version update varies on the [semantic](#) versioning of your packages. For example , a `^` caret would only update to the latest minor version.

If you need Poetry to update to a specific version, you can run a command `poetry add`, passing the package name with the version as parameter.

```
1. poetry add "uvicorn==0.17.5"
```

(Or if your package accept extra parameters during installation)

```
1. poetry add "uvicorn[standard]==0.17.5"
```

You might be wondering why are we using `poetry add` here? Well, Poetry is smart enough to detect if the package already exists in your system. It does not mean that it will keep two different versions of the package. But instead, it will replace the old version with the new one. At the same time, by running the said command, it will also update all its chain of dependencies. You will also notice that these changes are also reflected in the files `pyproject.toml` and `poetry.lock`. Now we're rolling!

Tips: After you update individually the packages in `pyproject.toml`, run again a mass update. This will allow poetry to check and update the remaining child dependencies.

```
1. poetry update
```

- Installing `pytest-asyncio` (0.18.1)
- Installing `pytest-cov` (3.0.0)
- Installing `pytest-mock` (3.7.0)
- Installing `python-json-logger` (2.0.2)
- Installing `pytz` (2021.3)
- Installing `sentry-asgi` (0.2.0)
- Installing `starlette-context` (0.3.3)
- Installing `uvicorn` (0.17.5)

Which Package Depends to What?

When you install a package in your system, most of the time the package itself comes with its own dependencies. Poetry has a way to list the dependencies as a tree using again `poetry show` but this time with a different flag.

```
1. poetry show -t
```

uvicorn 0.17.5

The lightning-fast ASGI server.

— asgiref >=3.4.0

— click >=7.0

- colorama *

— colorama >=0.4

— h11 >=0.8

— httptools >=0.2.0, <0.4.0

— python-dotenv >=0.13

— pyyaml >=5.1

— uvloop >=0.14.0, <0.15.0 || >0.15.0, <0.15.1 || >0.15.1

— watchgod >=0.6

— websockets >=10.0

This website uses cookies to improve your experience. By continuing to use YippeeCode.com you agree to its [Terms of Service](#).

ACCEPT

I advise that you look at the result of this command in your system. Sometimes you might be wondering why you are getting an error or it's not installing any packages when updating to the latest version. The reason could be is that one or more of your packages depends to a specific version of another package. For example above, `uvicorn` depends on `httptools` version greater than or equals to `0.2.0` but less than `0.4.0`. Also, to avoid breaking changes in your system, you should know up to what version you are allowed to upgrade a package.

Poetry SolverProblemError during update

Because project-x depends on both mypy (0.931) and mypy (0.910), version solving failed.

```
at ~/.poetry/lib/poetry/puzzle/solver.py:241 in _solve
237 |         packages = result.packages
238 |     except OverrideNeeded as e:
239 |         return self.solve_in_compatibility_mode(e.overrides, use_latest=use_latest)
240 |     except SolveFailure as e:
→ 241 |         raise SolverProblemError(e)
242 |
243 |     results = dict(
244 |         depth_first_search(
245 |             PackageNode(self._package, packages), aggregate_package_nodes
```

This error happens when you try to update a package that is a dependency in development without passing the `--dev` flag. If you remember, without this flag, Poetry will add the package in `tool.poetry.dependencies` section of `pyproject.toml` which is the list of packages that gets deployed in production use. If the package that you are trying to update is in `tool.poetry.dev-dependencies` section, make sure to add the flag.

```
1. | poetry add "mypy==0.931" --dev
```

Poetry EnvCommand / FileNotFoundError Error during update

```
, line 2127, in distributions_from_metadata
    if len(os.listdir(path)) == 0:
FileNotFoundError: [Errno 2] No such file or directory: '/Use
e-packages/~sa-4.7.2.dist-info'
```

```
at ~/.poetry/lib/poetry/utils/env.py:1074 in _run
1070 |         output = subprocess.check_output(
1071 |             cmd, stderr=subprocess.STDOUT,
1072 |         )
1073 |     except CalledProcessError as e:
→ 1074 |         raise EnvCommandError(e, input=input_)
1075 |
1076 |     return decode(output)
1077 |
1078 |     def execute(self, bin, *args, **kwargs):
```

While/After updating packages in my system, I encountered some weird error saying “No such file or directory”. After some googling, I found a [discussion](#) about this error years ago. Some says it could be related to the virtual environment. But to be honest, these errors are still cryptic to me. Nevertheless, I wanted to share a possible fix that works for me (try at your own risk).

First, I make sure that my lock file `poetry.lock` is updated.

```
1. | poetry lock
```

Second, I try to run `poetry update` again. If you still get an error, it might be a good idea to rebuild your virtual environment. Make sure to update your lock file first.

```
1. | poetry env list --full-path
2. | poetry env remove rex-XRBsn-py3.8
3. | poetry env use $(which python3)
4. | source $(dirname $(poetry run which python3))/activate
```

This website uses cookies to improve your experience. By continuing to use YippeeCode.com you agree to its [Terms](#)

[of Service](#). ACCEPT

- The first command is for listing your current virtual environment.
- Once you get the name of your environment, the second command is for removing this.
- The third command is for creating a new virtual environment.
- And finally, activating your new environment again.

Once you have a brand new environment, run an install.

```
1. poetry install
```

Recap of all the commands for updating packages in Python Poetry

For quick reference, here is a recap of all the commands I mentioned in this tutorial Update packages in Python Poetry.

```
1. poetry show -l
2. poetry show -t
3. poetry add "package==version"
4. poetry add "package[extra]==version"
5. poetry add "package==version" --dev
6. poetry update
7. poetry env list --full-path
8. poetry env remove name-of-env
9. poetry env use $(which python3)
10. source $(dirname $(poetry run which python3))/activate
```

Related Posts:

1. [Python Poetry Cheat Sheet](#)
2. [Upgrading to Django 3 and Python 3](#)
3. [Python unittest tips and tricks](#)
4. [Quick guide to Asyncio in Python](#)

Category: [Featured](#), [Programming](#), [Tutorials](#)
Tags: [package manager](#), [pip](#), [poetry](#), [python](#)

© YippeeCode.com 2020